Kazi Sadat Hossain

kazisadath@gmail.com

Project Title: Taxi Booking With Fare Prediction

Objective:
The objective of this project is to develop an interactive Taxi Booking System with fare prediction that allows users to book taxis efficiently and conveniently. The system provides a user-friendly interface where customers can select pickup and drop locations and automatically be assigned an available taxi. It maintains a real-time status of taxis, indicating which vehicles are available or busy, and calculates the estimated fare for each ride based on distance and predefined pricing rules. Additionally, the system keeps a detailed ride history for users, including driver details, taxi information, distance traveled, and fare. Overall, this project demonstrates the practical application of database management, PHP programming, and dynamic web development to create a functional transportation management system that enhances both user experience and taxi management efficiency.

Problem Statement:
Urban transportation services require an efficient and automated system to manage taxi bookings, vehicle availability, fare calculations, and ride records. In many traditional or manually maintained systems, taxis may be booked inefficiently, their availability status is not updated in real-time, and fare estimation often relies on manual calculations or approximate values. These shortcomings can result in delays for passengers, underutilized or overbooked taxis, and inconsistent fare charges, reducing overall service reliability and user satisfaction. Moreover, without a well-structured database linking users, bookings, taxis, and locations, it becomes difficult to maintain accurate records of rides and fares. Thus, an automated taxi booking system with real-time taxi management and accurate fare prediction is essential to streamline ride assignments, provide reliable fare information, and ensure consistent, efficient, and user-friendly service.

Entity Relationship Diagram:

The ERD of the Taxi System illustrates the relationships between Users, Bookings, Taxis, and Locations, representing the core structure of the database for managing taxi bookings and fare predictions.

Entities and Attributes

Users:

- user_id (PK): Unique identifier for each user.
- name: Name of the user.
- email: Email of the user, used for login.
- password: Hashed password for authentication.

Represents the passengers who book rides through the system.

Taxis:

- taxi_id (PK): Unique identifier for each taxi.
- driver_name: Name of the taxi driver.
- plate_no: Registration number of the taxi.

- status: Indicates whether the taxi is Available or Busy.

Manages taxi availability and driver information for ride allocation.

Locations:

- location_id (PK): Unique identifier for each location.
- name: Name of the location (e.g., Mirpur, Banani).
- distance_km: Distance of the location from a base point, used for fare calculation.

Stores both pickup and drop-off points for rides. The same table is used for both types of locations to reduce redundancy.

Bookings:

- booking_id (PK): Unique identifier for each booking.
- user_id (FK): References Users.user_id to identify the passenger.
- taxi_id (FK): References Taxis.taxi_id to identify the assigned taxi.
- pickup_loc (FK): References Locations.location_id for the pickup point.
- drop_loc (FK): References Locations.location_id for the drop-off point.
- distance_km: Calculated distance of the ride.
- fare: Calculated fare for the ride.
- booking_time: Timestamp of when the booking was made.
- ride_status: marks ride as either ‘complete’, ‘cancelled’, ‘pending’ (enum values)

Records all ride transactions, linking users, taxis, and locations while storing ride details such as distance and fare.

Relationships

1.Users ↔ Bookings:
One-to-Many (1:N) relationship: Each user can have multiple bookings, but each booking belongs to only one user.

2.Taxis ↔ Bookings:
One-to-Many (1:N) relationship: Each taxi can serve multiple bookings over time, but each booking is assigned to only one taxi.

3.Locations ↔ Bookings (Pickup and Drop):
One-to-Many (1:N) relationship: Each location can serve as a pickup or drop-off point for multiple bookings.

Design Highlights

The Bookings table serves as the central entity, connecting Users, Taxis, and Locations, which enables the

system to track rides efficiently. Using a single Locations table for both pickup and drop locations reduces redundancy and ensures consistency. Taxi status management (Available/Busy) supports automated ride allocation and helps visualize taxi availability in the dashboard. Calculating distance and fare based on location distances allows the system to provide fare prediction in real time.

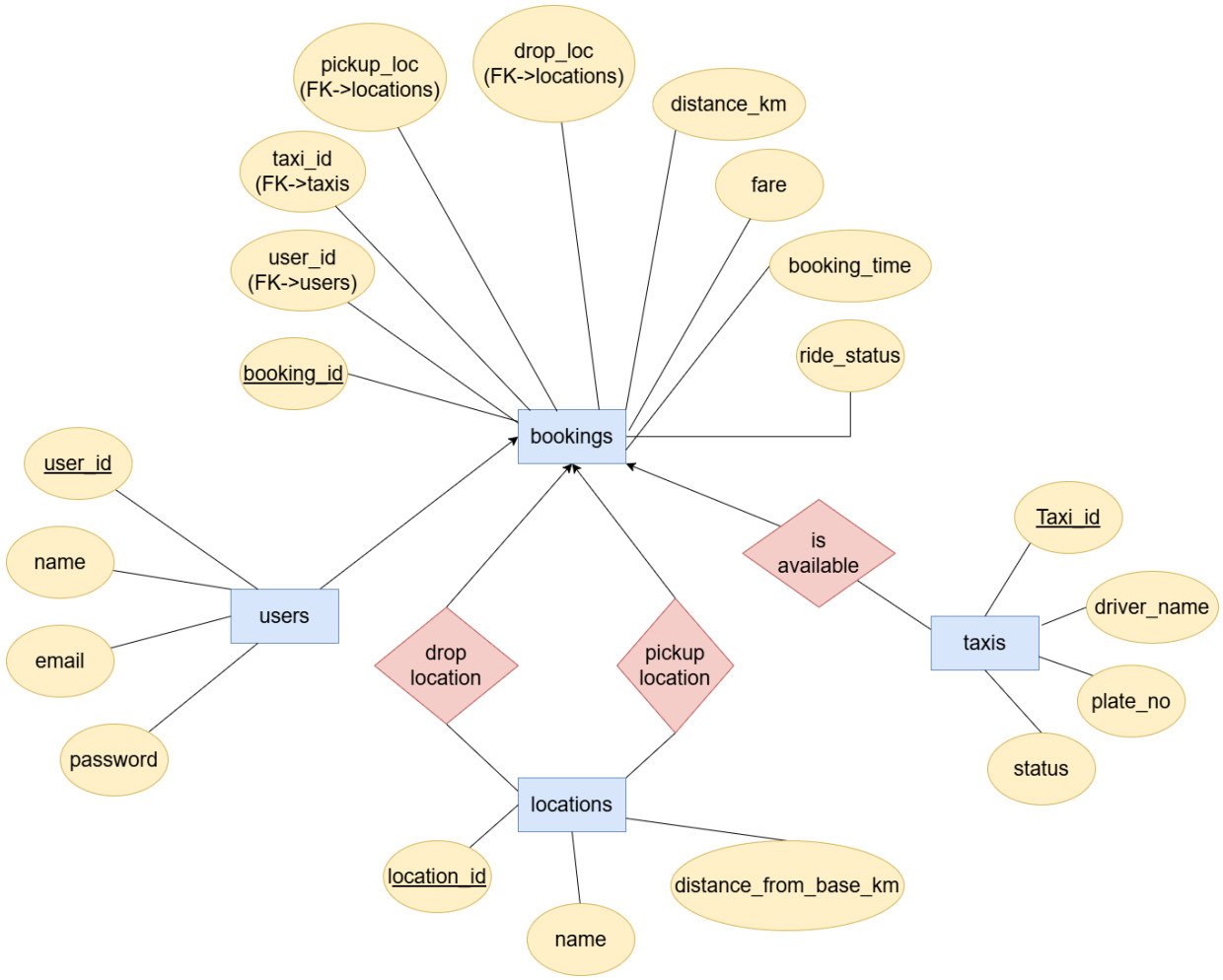

Fig 01: Entity Relationship Diagram

MySQL and PHP Integration

The Taxi System with Fare Prediction uses PHP as the server-side scripting language and MySQL as the relational database management system to handle all data storage and retrieval operations. PHP scripts interact with the MySQL database to perform essential functions such as user authentication, taxi allocation, fare calculation, and ride history management. Using mysql functions or PHP Data Objects (PDO), the system executes SQL queries to insert, update, and fetch data from tables like Users, Taxis, Locations, and Bookings. This integration enables real-time updates, such as changing taxi status to “Busy” when a ride is booked, calculating fares dynamically based on distance, and maintaining a detailed history of user rides.

The development and testing of the system were performed using XAMPP, an integrated development environment that includes Apache (web server), MySQL (database), and PHP. XAMPP provides a local server environment, allowing seamless execution of PHP scripts, database management, and front-end testing in a single platform. This setup simplifies development, ensures compatibility between PHP and MySQL, and allows rapid testing of the application before deployment.

Database Design

The database for the Taxi System with Fare Prediction consists of four main tables: Users, Taxis, Locations, and Bookings. Each table is carefully structured to maintain data integrity, enforce logical constraints, and support efficient taxi operations, automated fare calculation, and ride tracking.

a) users Table
Stores passenger information and enables authentication for the system.

Attributes:
user_id – INT, PRIMARY KEY, AUTO_INCREMENT
name – VARCHAR(100)
email – VARCHAR(100), UNIQUE
password – VARCHAR(255)

Constraints:
user_id serves as the primary key.
email must be unique to prevent duplicate accounts.
password is securely stored for authentication.

b)taxis Table
Maintains information about taxis and their drivers, and tracks taxi availability.

Attributes:
taxi_id – INT, PRIMARY KEY, AUTO_INCREMENT
driver_name – VARCHAR(100)
plate_number – VARCHAR(50)
status – ENUM('Available', 'Busy'), default 'Available'

Constraints:
taxi_id is the primary key.
status is restricted to predefined values to ensure accurate tracking of taxi availability.

c) locations Table
Stores all potential pickup and drop-off points, along with distance from a reference base, used for fare calculation.

Attributes:
location_id – INT, PRIMARY KEY, AUTO_INCREMENT

name – VARCHAR(100)
distance_from_base_km – FLOAT
Constraints:
location_id is the primary key.
distance_from_base_km must be a non-negative value.

d) bookings Table
Connects users, taxis, and locations to record ride transactions, including fare, distance, and timestamp.

Attributes:
booking_id – INT, PRIMARY KEY, AUTO_INCREMENT
user_id – INT, FOREIGN KEY REFERENCES users(user_id)
taxi_id – INT, FOREIGN KEY REFERENCES taxis(taxi_id)
pickup_location_id – INT, FOREIGN KEY REFERENCES locations(location_id)
drop_location_id – INT, FOREIGN KEY REFERENCES locations(location_id)
distance_km – FLOAT
fare – FLOAT
booking_time – TIMESTAMP, default CURRENT_TIMESTAMP
ride_status – ENUM('Complete', 'Cancel', 'Pending'), default 'Pending'

Constraints:
booking_id is the primary key.
user_id must exist in the Users table.
taxi_id must exist in the Taxis table.
pickup_location_id and drop_location_id must exist in the Locations table.
distance_km and fare must be positive values.

This design ensures data integrity through foreign key constraints, supports real-time taxi allocation, and enables automated fare calculation based on the distance between locations. The centralized Bookings table allows the system to maintain ride history, track taxi utilization, and analyze operational efficiency. Overall, this relational structure provides a scalable, reliable, and efficient foundation for an automated taxi booking and fare prediction system.

SQL queries and output

1.Database Creation

```
CREATE DATABASE taxi_system;
USE taxi_system;

CREATE TABLE users (
   user_id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(100),
   email VARCHAR(100) UNIQUE,
   password VARCHAR(255)
```

```
);

CREATE TABLE locations (
   location_id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(100),
   distance_from_base_km FLOAT
);


INSERT INTO locations (name, distance_from_base_km) VALUES
("Mirpur", 2),
("Banani", 5),
("Gulshan", 6),
("Uttara", 10),
("Dhanmondi", 4),
("Motijheel", 7);

CREATE TABLE taxis (
   taxi_id INT AUTO_INCREMENT PRIMARY KEY,
   driver_name VARCHAR(100),
   plate_number VARCHAR(50),
   status ENUM('Available','Busy') DEFAULT 'Available'
);

INSERT INTO taxis (driver_name, plate_number, status) VALUES
("Rahim Uddin", "DHA-11-1234", "Available"),
("Karim Sheikh", "DHA-77-9876", "Busy"),
("Selim Miah", "DHA-44-5678", "Available"),
("Jamal Hossain", "DHA-22-4455", "Available");

CREATE TABLE bookings (
   booking_id INT AUTO_INCREMENT PRIMARY KEY,
   user_id INT,
   taxi_id INT,
   pickup_location_id INT,
   drop_location_id INT,
   distance_km FLOAT,
   fare FLOAT,
   booking_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

   FOREIGN KEY (user_id) REFERENCES users(user_id),
   FOREIGN KEY (taxi_id) REFERENCES taxis(taxi_id),
   FOREIGN KEY (pickup_location_id) REFERENCES locations(location_id),
   FOREIGN KEY (drop_location_id) REFERENCES locations(location_id)
);
```

```
ALTER TABLE bookings
ADD COLUMN ride_status ENUM('pending', 'cancelled', 'completed')
NOT NULL DEFAULT 'pending';
```

```
MariaDB [(none)]> use taxi_system;
Database changed
MariaDB [taxi_system]> show tables;
+-----------------------+
| Tables_in_taxi_system |
+-----------------------+
| bookings              |
| locations             |
| taxis                 |
| users                 |
+-----------------------+
4 rows in set (0.001 sec)

MariaDB [taxi_system]>
```

Fig 02: Database & Table creation

PHP code execution and MySQL integration:

2. db.php

```php
<?php
$host = "localhost";
$user = "root";
$pass = "";
$db = "taxi_system";
$port = 3307;

$conn = new mysqli($host, $user, $pass, $db, $port);

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
?>
```

The db.php file serves as the central database connection script for the Taxi System. It establishes a connection between the PHP application and the MySQL database using defined credentials, including host,

username, password, database name, and port.

Done by- Kazi Sadat Hossain

3. header.php

```
<!DOCTYPE html>
<html>
<head>
    <title>Taxi System</title>
    <style>
        body { font-family: Arial; margin: 30px; }
        .box { width: 300px; padding: 20px; border: 1px solid #ccc;
border-radius: 5px; margin:auto; }
        .btn { padding: 7px 20px; background:black; color:white; border:none;
cursor:pointer; }
        table, td, th { border:1px solid #444; border-collapse: collapse;
padding:8px; }
    </style>
</head>
<body>
```

This portion of the code helps to create uniform layout and styling for the front-end portion.

Done by- Zarin Tasnim

4. Login

a) Login Frontend

```
<?php include "header.php"; ?>
<hr>
<h2>User Login</h2>

<div class="box">
<form action="login_process.php" method="POST">

    <label>Email:</label><br>
    <input type="email" name="email" required><br><br>
```

```
    <label>Password:</label><br>
    <input type="password" name="password" required><br><br>

    <button class="btn" type="submit">Login</button>

</form>

<p>Don't have an account? <a href="register.php">Register</a></p>
</div>
<hr>
```

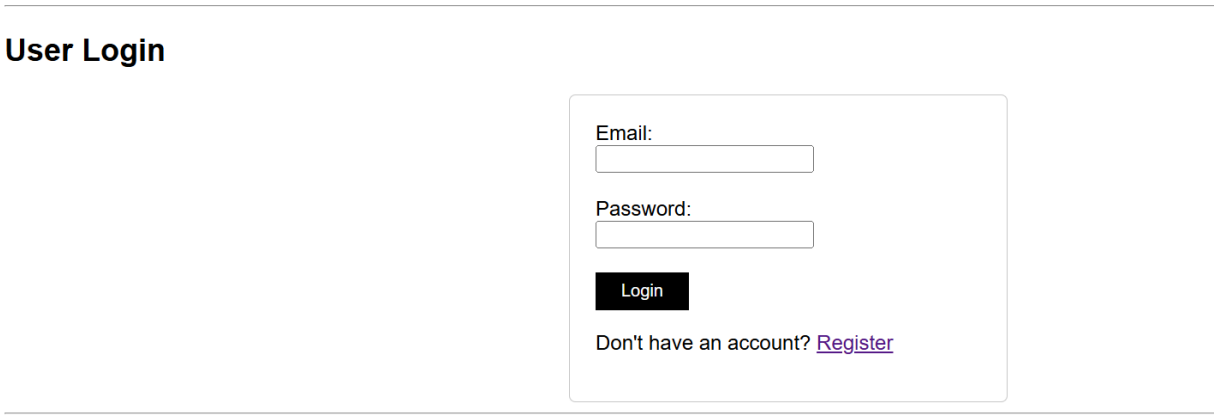

Fig 03:Login Page (Front End)

Done by- zarin Tasnim

b) Login logic

```
<?php
session_start();
include "db.php";


$email = $_POST['email'] ?? '';
$pass  = $_POST['password'] ?? '';

if (empty($email) || empty($pass)) {
```

```
    die("Please enter email and password.");
}

$sql = "SELECT user_id, name, password FROM users WHERE email='$email' LIMIT
1";
$res = $conn->query($sql);

if ($res->num_rows === 1) {
    $row = $res->fetch_assoc();

    if (password_verify($pass, $row['password'])) {
        $_SESSION['user_id'] = $row['user_id'];
        $_SESSION['user_name'] = $row['name'];

        echo "Login successful!";
        header("Location: dashboard.php");

    } else {
        echo "Invalid password.";
    }
} else {
    echo "No account found with this email.";
}
?>
```

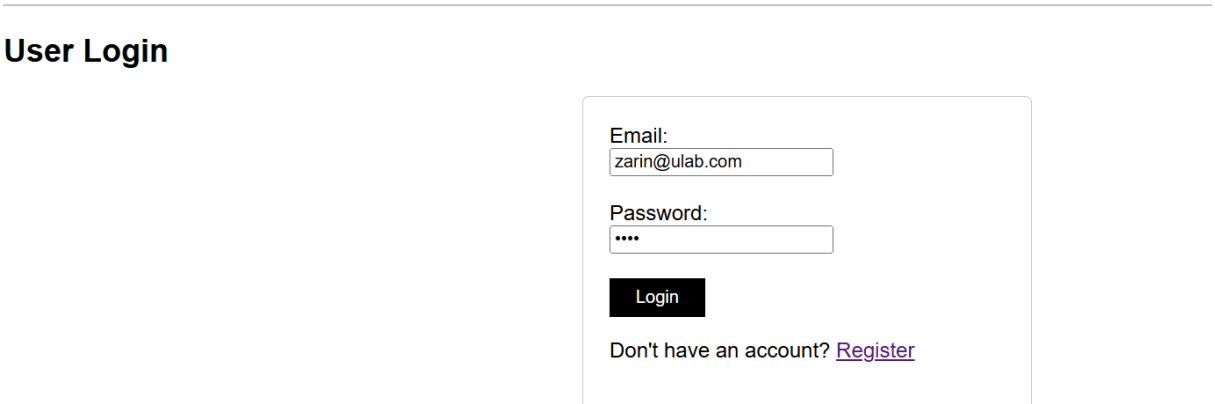

Fig 04:Login Page logic

```
MariaDB [taxi_system]> select * from users;
+---------+---------------+-----------------+-------------------------------------------------------------+
| user_id | name          | email           | password                                                    |
+---------+---------------+-----------------+-------------------------------------------------------------+
|       1 | Zarin Tasnim  | zarin@ulab.com  | $2y$10$C0rlwoLFmvfrqRUDxrmcSOXM6pq3I/JTDrNAICqeoWf9VnwC6fIwi |
|       2 | Sadat Hossain | sadat@gmail.com | $2y$10$ejaDAuRBRabNjps3J1DwT.ttN4k9EeY3SAzjp1SUzhM8mhJH5sm/6 |
+---------+---------------+-----------------+-------------------------------------------------------------+
2 rows in set (0.001 sec)
```

Fig 05: Database user integration

The login functionality in the Taxi System is divided into two key components: login.php, which provides the front-end interface for user authentication, and loginprocess.php, which handles the back-end processing of login credentials.

Done by- Kazi Sadat Hossain

5. Register

a) register.php (Frontend)

```php
<?php include "header.php"; ?>
<hr>
<h2>User Registration</h2>

<div class="box">
<form action="register_process.php" method="POST">
    <label>Name:</label><br>
    <input type="text" name="name" required><br><br>

    <label>Email:</label><br>
    <input type="email" name="email" required><br><br>

    <label>Password:</label><br>
    <input type="password" name="password" required><br><br>

    <button class="btn" type="submit">Register</button>
</form>

<p>Already have an account? <a href="login.php">Login</a></p>
</div>

<hr>
```

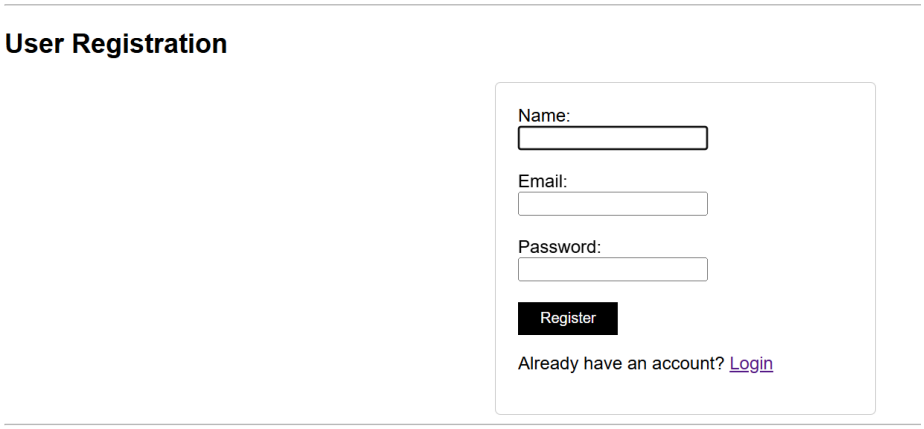

Fig 06: Register page Frontend

Done by- Zarin Tasnim

b) register_process.php

```
<?php
include "db.php";

$name = $_POST['name'];
$email = $_POST['email'];
$password = password_hash($_POST['password'], PASSWORD_DEFAULT);

$sql = "INSERT INTO users (name, email, password) VALUES ('$name', '$email',
'$password')";

if ($conn->query($sql)) {
    header("Location: login.php");
} else {
    echo "Error: " . $conn->error;
}
?>
```

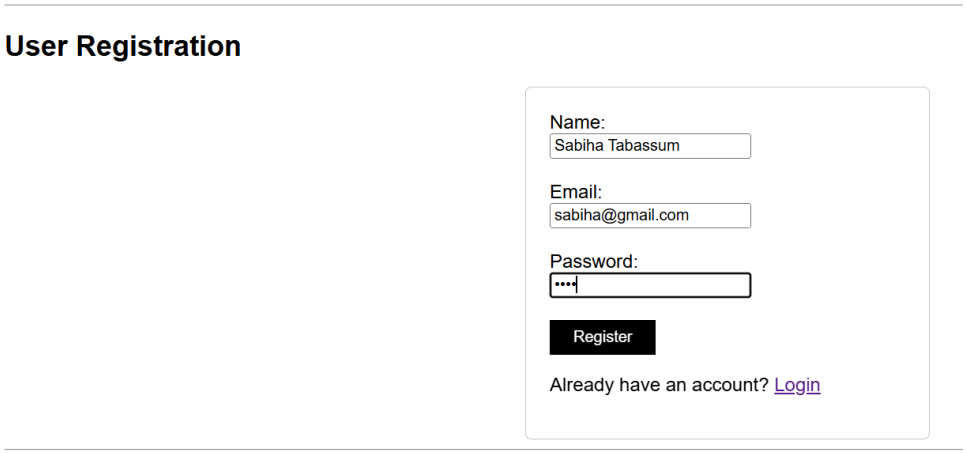

Fig 07: register logic

```
MariaDB [taxi_system]> select * from users;
+---------+----------------+------------------+-------------------------------------------------------------+
| user_id | name           | email            | password                                                    |
+---------+----------------+------------------+-------------------------------------------------------------+
|       1 | Zarin Tasnim   | zarin@ulab.com   | $2y$10$C0rlwoLFmvfrqRUDxrmcSOXM6pq3I/JTDrNAICqeoWf9VnwC6fIwi |
|       2 | Sadat Hossain  | sadat@gmail.com  | $2y$10$ejaDAuRBRabNjps3J1DwT.ttN4k9EeY3SAzjp1SUzhM8mhJH5sm/6 |
|       3 | Sabiha Tabassum | sabiha@gmail.com | $2y$10$bjB1lsJBoBdGc0N7nXW3ne3iLmzQO6R4UJYkpXhkONEXyJmRz7Jau |
+---------+----------------+------------------+-------------------------------------------------------------+
3 rows in set (0.000 sec)
```

Fig 08: Database register page integration

The registration functionality in the Taxi System has two main parts. register.php provides a form where new users can enter their name, email, and password to create an account. registerprocess.php handles the back-end, securely storing the user's information in the database and checking for duplicate emails to prevent multiple accounts with the same email address. This separation ensures that account creation is secure, organized, and easy for users to complete.

Done by- Kazi Sadat Hossain

6. dashboard.php

```php
<?php
session_start();
if (!isset($_SESSION['user_id'])) {
    header("Location: login.php");
    exit;
}
include "db.php";
include "header.php";
?>
```

```
<hr>
<br><br>
<h2>Welcome, <?php echo $_SESSION['user_name']; ?></h2>
<div style="text-align: right;">
    <a href="help.php" class="btn">Help</a>
</div>
<br><br>
<hr>

<h3>Book a Ride</h3>

<form action="book.php" method="POST">

    <label>From:</label><br>
    <select name="pickup" required>
        <?php
        $loc = $conn->query("SELECT * FROM locations");
        while ($row = $loc->fetch_assoc()) {
            echo "<option
value='{$row['location_id']}'>{$row['name']}</option>";
        }
        ?>
    </select><br><br>

    <label>To:</label><br>
    <select name="drop" required>
        <?php
        $loc2 = $conn->query("SELECT * FROM locations");
        while ($row = $loc2->fetch_assoc()) {
            echo "<option
value='{$row['location_id']}'>{$row['name']}</option>";
        }
        ?>
    </select><br><br>

    <button class="btn">Book Taxi</button>
</form>
<br><br>
```

```
    <a href="ride.php" class="btn">See current ride</a>
<br><br>
<hr>
<h3>Available Taxis</h3>
<table>
<tr>
    <th>ID</th>
    <th>Driver</th>
    <th>Plate</th>
</tr>

<?php
$available = $conn->query("SELECT * FROM taxis WHERE status='Available'");
while ($t = $available->fetch_assoc()) {
    echo "<tr>
            <td>{$t['taxi_id']}</td>
            <td>{$t['driver_name']}</td>
            <td>{$t['plate_number']}</td>
          </tr>";
}
?>
</table>
<h3>Busy Taxis (On Ride)</h3>

<table>
<tr>
    <th>ID</th>
    <th>Driver</th>
    <th>Plate</th>
</tr>

<?php
$busy = $conn->query("SELECT * FROM taxis WHERE status='Busy'");
while ($t = $busy->fetch_assoc()) {
    echo "<tr>
            <td>{$t['taxi_id']}</td>
            <td>{$t['driver_name']}</td>
            <td>{$t['plate_number']}</td>
          </tr>";
```

```
}
?>
</table>

<br>
<hr>
<br><br>

<a href="records.php" class="btn">View Ride history</a>

<br><br>
<hr>
<br>
<a href="logout.php" class="btn">Logout</a>
<br><br>
<hr>
<br>
<a href="logout.php" class="btn">Logout</a>
```

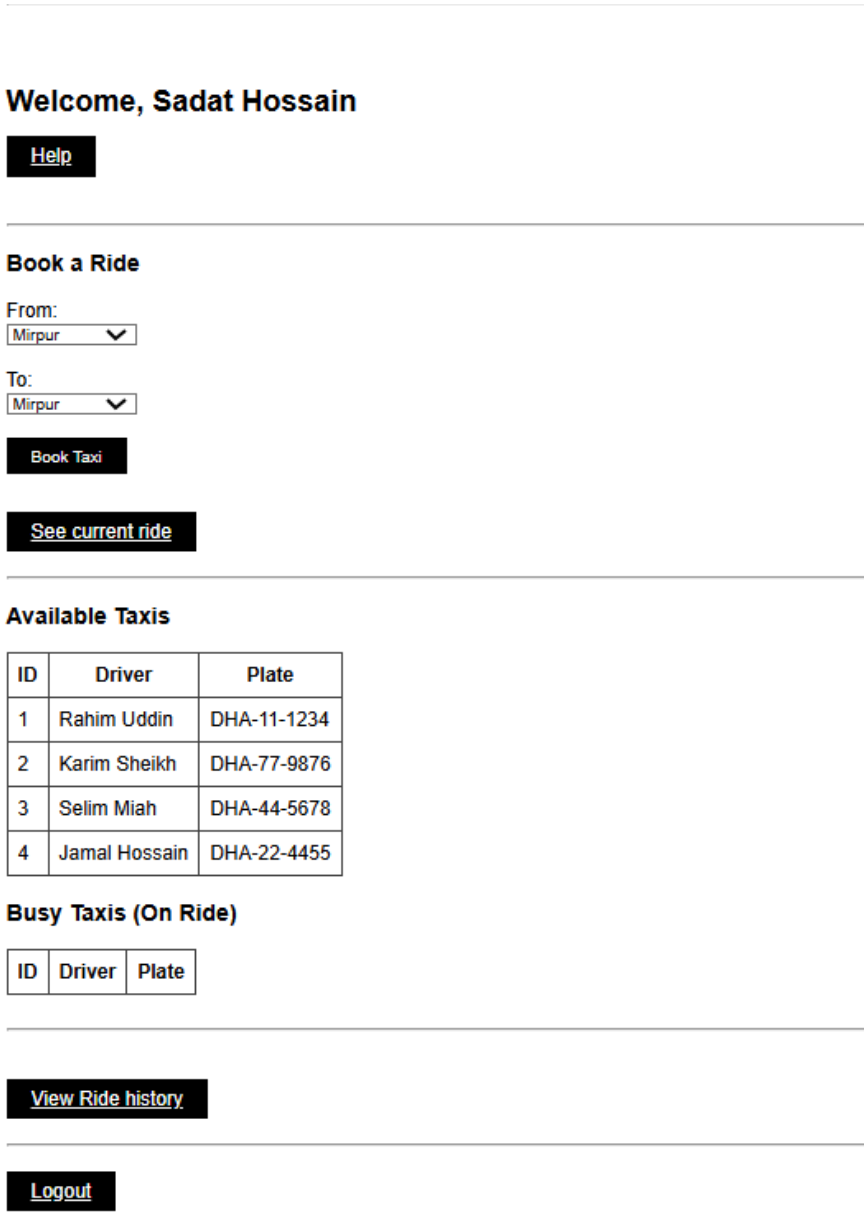

Fig 09:User dashboard

The dashboard.php page serves as the central interface for logged-in users in the Taxi System. It allows users to book rides by selecting pickup and drop-off locations, view the list of available taxis along with their status, and access links to their ride history or logout. The page dynamically retrieves data from the database, such as taxi availability and location options, ensuring that users always have up-to-date information for making booking decisions. By consolidating ride booking, taxi status monitoring, and navigation to other functionalities in one interface, the dashboard provides a user-friendly and interactive experience, enabling efficient management of rides within the system.Fig 09: user Dashboard

Done by- Kazi Sadat Hossain

7. Book.php

```php
<?php
session_start();
include "db.php";

if (!isset($_SESSION['user_id'])) {
    header("Location: login.php");
    exit;
}

$user = (int)$_SESSION['user_id'];
$pickup = (int)$_POST['pickup'];
$drop   = (int)$_POST['drop'];

if ($pickup === $drop) {
    die("Pickup and Drop location cannot be the same.");
}


$pickupData = $conn->query("SELECT distance_from_base_km FROM locations WHERE
location_id = $pickup")->fetch_assoc();
$dropData   = $conn->query("SELECT distance_from_base_km FROM locations WHERE
location_id = $drop")->fetch_assoc();

$distance = abs($pickupData['distance_from_base_km'] -
$dropData['distance_from_base_km']);
$baseFare = 50;
$perKmRate = 20;
$fare = $baseFare + ($distance * $perKmRate);

try {

    $conn->begin_transaction();

    $taxiResult = $conn->query("SELECT taxi_id FROM taxis WHERE
status='Available' ORDER BY taxi_id LIMIT 1 FOR UPDATE");

    if ($taxiResult->num_rows === 0) {
        throw new Exception("Sorry, no taxis are currently available.");
    }
```

```
    $taxi = $taxiResult->fetch_assoc()['taxi_id'];


    $stmt = $conn->prepare("
        INSERT INTO bookings (user_id, taxi_id, pickup_location_id,
drop_location_id, distance_km, fare)
        VALUES (?, ?, ?, ?, ?, ?)
    ");
    $stmt->bind_param("iiiidd", $user, $taxi, $pickup, $drop, $distance,
$fare);
    $stmt->execute();


    $conn->query("UPDATE taxis SET status='Busy' WHERE taxi_id = $taxi");


    $conn->commit();


    $taxiInfo = $conn->query("SELECT driver_name, plate_number FROM taxis
WHERE taxi_id = $taxi")->fetch_assoc();
    echo "<h3>Booking Successful!</h3>";
    echo "Driver: {$taxiInfo['driver_name']}<br>";
    echo "Taxi License Plate: {$taxiInfo['plate_number']}<br>";
    echo "Taxi ID: $taxi<br>";
    echo "Distance: $distance km<br>";
    echo "Fare: $fare BDT<br><br>";
    echo "<a href='dashboard.php'>Back to Dashboard</a>";

} catch (Exception $e) {
    $conn->rollback();
    die($e->getMessage());
}
?>
```

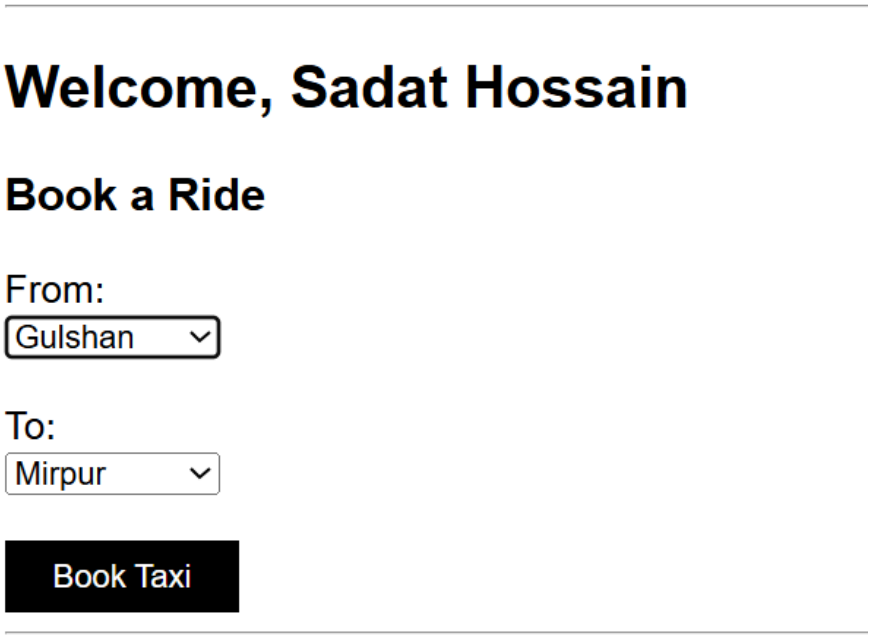

Fig 10: Booking a taxi

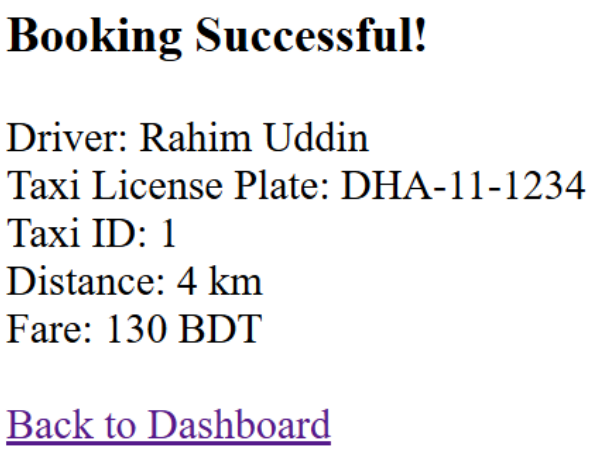

Fig 11: booking confirmation

The book.php file handles the ride booking process for the Taxi System. It serves as the back-end script that receives the user's selected pickup and drop-off locations from the dashboard form. The script calculates the distance between the selected locations and estimates the fare based on a defined base fare, per-kilometer rate, and any additional service charges. It then identifies an available taxi from the database, assigns it to the user, and updates the taxi's status to "Busy" to prevent double-booking. Once the booking is successfully saved in the Bookings table, book.php displays a confirmation message including the taxi ID, driver name, license plate, distance, and estimated fare. By automating taxi allocation and fare calculation, this script ensures a seamless, real-time booking experience while maintaining accurate ride records for both users and taxis.

Done by- Kazi Sadat Hossain

7. records.php

```
<?php
session_start();
if (!isset($_SESSION['user_id'])) {
    header("Location: login.php");
    exit;
}

include "db.php";
include "header.php";
?>

<h2>Your Ride History</h2>
<a href="dashboard.php" class="btn">Back to Dashboard</a>
<br><br>
<hr>
<table border="1" cellpadding="8" cellspacing="0">
<tr>
    <th>Pickup</th>
    <th>Drop</th>
    <th>Driver</th>
    <th>License Plate</th>
    <th>Distance (km)</th>
    <th>Fare (BDT)</th>
    <th>Time</th>
    <th>Ride Status</th>
</tr>

<?php
$uid = $_SESSION['user_id'];

$history = $conn->query("
    SELECT l1.name AS pickup, l2.name AS
drop_loc,t.driver_name,t.plate_number AS license_plate, b.distance_km,
b.fare, b.booking_time,b.ride_status
    FROM bookings b
    JOIN locations l1 ON b.pickup_location_id = l1.location_id
    JOIN locations l2 ON b.drop_location_id = l2.location_id
```

```
    JOIN taxis t ON b.taxi_id = t.taxi_id
    WHERE b.user_id = $uid
    ORDER BY b.booking_time DESC
");

while ($r = $history->fetch_assoc()) {
    echo "<tr>
        <td>{$r['pickup']}</td>
        <td>{$r['drop_loc']}</td>
        <td>{$r['driver_name']}</td>
        <td>{$r['license_plate']}</td>
        <td>{$r['distance_km']}</td>
        <td>{$r['fare']}</td>
        <td>{$r['booking_time']}</td>
        <td>{$r['ride_status']}</td>
    </tr>";
}
?>
</table>
<hr>
```

Your Ride History

Back to Dashboard

Pickup	Drop	Driver	License Plate	Distance (km)	Fare (BDT)	Time	Ride Status
Uttara	Dhanmondi	Karim Sheikh	DHA-77-9876	6	170	2025-12-16 13:16:57	completed
Banani	Motijheel	Rahim Uddin	DHA-11-1234	2	90	2025-12-16 13:11:09	completed
Banani	Motijheel	Rahim Uddin	DHA-11-1234	2	90	2025-12-16 13:07:17	cancelled
Banani	Motijheel	Jamal Hossain	DHA-22-4455	2	90	2025-12-14 01:27:37	completed
Banani	Dhanmondi	Selim Miah	DHA-44-5678	1	70	2025-12-14 01:27:32	completed
Gulshan	Mirpur	Karim Sheikh	DHA-77-9876	4	130	2025-12-14 01:27:24	cancelled

Fig 12: recording past taxi booking

The ride history functionality displays a detailed record of all rides booked by a user. It retrieves data from the Bookings, Taxis, and Locations tables to provide comprehensive information for each ride, including pickup and drop-off locations, driver name, taxi license plate, distance traveled, fare, and booking time. By joining these tables, the system ensures that users can see accurate, real-time details of their previous rides in a structured tabular format. This feature enhances transparency and allows users to review their ride patterns, costs, and taxi assignments, contributing to a more informative and user-friendly experience within the Taxi

System.

Done by- Kazi Sadat Hossain

8.ride.php

```
<?php
session_start();
if (!isset($_SESSION['user_id'])) {
    header("Location: login.php");
    exit;
}

include "db.php";
include "header.php";

$uid = $_SESSION['user_id'];

if (isset($_POST['booking_id'], $_POST['status'])) {

    $booking_id = (int)$_POST['booking_id'];
    $status     = $_POST['status'];

    if ($status === 'completed' || $status === 'cancelled') {

        $conn->begin_transaction();

        try {
            $stmt = $conn->prepare("
                UPDATE bookings
                SET ride_status = ?
                WHERE booking_id = ? AND user_id = ? AND ride_status =
'pending'
            ");
            $stmt->bind_param("sii", $status, $booking_id, $uid);
            $stmt->execute();

            if ($stmt->affected_rows > 0) {
                $stmt2 = $conn->prepare("
                    UPDATE taxis t
```

```
                    JOIN bookings b ON t.taxi_id = b.taxi_id
                    SET t.status = 'Available'
                    WHERE b.booking_id = ? AND b.user_id = ?
                ");
                $stmt2->bind_param("ii", $booking_id, $uid);
                $stmt2->execute();
                $stmt2->close();
            }

            $stmt->close();
            $conn->commit();

        } catch (Exception $e) {
            $conn->rollback();
            die("Error updating ride: " . $e->getMessage());
        }
    }
}
?>

<h2>Your Current Ride</h2>
<a href="dashboard.php" class="btn">Back to Dashboard</a>
<br><br>
<hr>

<table border="1" cellpadding="8" cellspacing="0">
<tr>
    <th>Pickup</th>
    <th>Drop</th>
    <th>Driver</th>
    <th>Plate</th>
    <th>Distance (km)</th>
    <th>Fare (BDT)</th>
    <th>Time</th>
    <th>Status</th>
    <th>Action</th>
</tr>

<?php
```

```
$current = $conn->prepare("
    SELECT b.booking_id,
           l1.name AS pickup,
           l2.name AS drop_loc,
           t.driver_name,
           t.plate_number,
           b.distance_km,
           b.fare,
           b.booking_time,
           b.ride_status
    FROM bookings b
    JOIN locations l1 ON b.pickup_location_id = l1.location_id
    JOIN locations l2 ON b.drop_location_id = l2.location_id
    JOIN taxis t ON b.taxi_id = t.taxi_id
    WHERE b.user_id = ? AND b.ride_status = 'pending'
    ORDER BY b.booking_time DESC
");

$current->bind_param("i", $uid);
$current->execute();
$result = $current->get_result();

if ($result->num_rows === 0) {
    echo "<tr><td colspan='9'>No active ride</td></tr>";
} else {
    while ($r = $result->fetch_assoc()) {
        echo "<tr>
            <td>{$r['pickup']}</td>
            <td>{$r['drop_loc']}</td>
            <td>{$r['driver_name']}</td>
            <td>{$r['plate_number']}</td>
            <td>{$r['distance_km']}</td>
            <td>{$r['fare']}</td>
            <td>{$r['booking_time']}</td>
            <td>{$r['ride_status']}</td>
            <td>
                <form method='post' style='display:inline'>
                    <input type='hidden' name='booking_id'
value='{$r['booking_id']}'>
```

```
                    <button class='btn' name='status'
value='completed'>Complete</button>
                </form>

                <form method='post' style='display:inline'>
                    <input type='hidden' name='booking_id'
value='{$r['booking_id']}'>
                    <button class='btn' name='status'
value='cancelled'>Cancel</button>
                </form>
            </td>
        </tr>";
    }
}

$current->close();
?>
</table>

<hr>
<br>
<a href="records.php" class="btn">View Ride History</a>
<br><br>
```

Your Current Ride

Back to Dashboard

Pickup	Drop	Driver	Plate	Distance (km)	Fare (BDT)	Time	Status	Action
Gulshan	Motijheel	Rahim Uddin	DHA-11-1234	1	70	2025-12-16 14:08:49	pending	Complete Cancel

View Ride History

Fig 13: recording current taxi booking

ride.php functions as a confirmation system for the users. When a ride is booked it will be shown in this page and later will receive the whether the ride was completed or cancelled after fetching the data it will populate the database accordingly. The code also make sures not to overlap the user bookings using transactions.

Done by- Kazi Sadat Hossain

9. help.php

```php
<?php
session_start();
if (!isset($_SESSION['user_id'])) {
    header("Location: login.php");
    exit;
}

include "header.php";
?>

<h2>Help & Account Settings</h2>
<a href="dashboard.php" class="btn">Back to Dashboard</a>

<hr>

<p>If you are facing issues or want to manage your account, you can use the
options below.</p>

<h3>Delete Account</h3>

<p style="color:red;">
    Warning: Deleting your account is permanent and cannot be undone.
</p>

<form action="delete_account.php" method="post"
      onsubmit="return confirm('Are you absolutely sure? This action cannot
be undone.');">
    <button class="btn">Delete My Account</button>
</form>

<hr>

<h3>Need Help?</h3>
<p>
    For support, please contact the administrator or check the user guide.
    Contact: olive-taxi_system@nomnom.com
</p>
```

```
<br>
<a href="logout.php" class="btn">Logout</a>
```

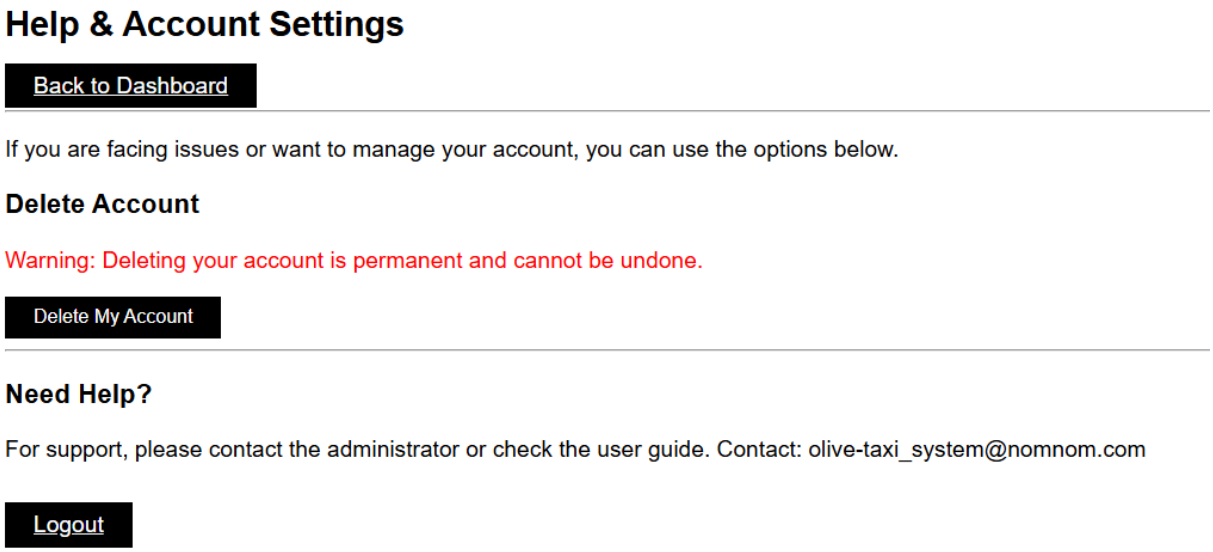

Fig 14: account deletion page

This page shows the option for account deletion and other help information such as contact info for the administration. The code for this page also give command the deletion of the current account in use.

Done by- Zarin Tasnim

10. delete_account.php

```
<?php
session_start();
if (!isset($_SESSION['user_id'])) {
    header("Location: login.php");
    exit;
}

include "db.php";

$uid = $_SESSION['user_id'];

$conn->query("
    UPDATE taxis t
```

```
    JOIN bookings b ON t.taxi_id = b.taxi_id
    SET t.status = 'Available'
    WHERE b.user_id = $uid
      AND b.ride_status = 'pending'
");

$conn->query("DELETE FROM bookings WHERE user_id = $uid");

$conn->query("DELETE FROM users WHERE user_id = $uid");


session_destroy();
header("Location: login.php");
exit;
```

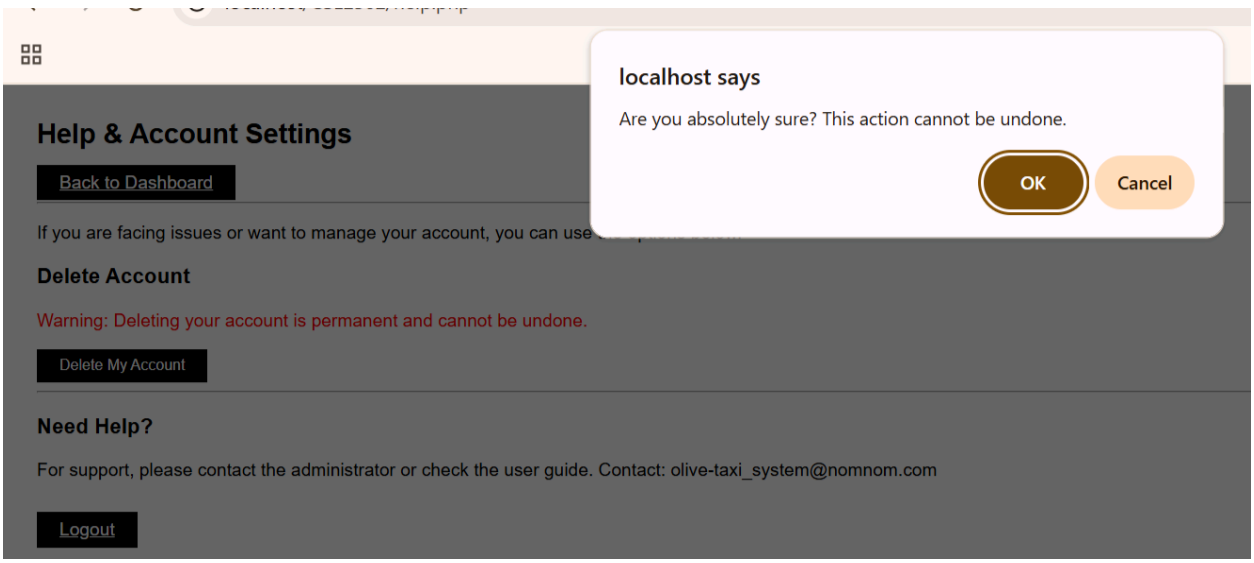

Fig 15: Account deletion confirmation

This page give confirmation page for the account deletion. After accepting the terms the user can delete their account. The code emphasizes on hard deletion meaning the user info is completely removed from the database. Since there is constraint where it uses a foreign key the command make sures take care of the constraints only after that the user information is deleted.

Done by- Zarin Tasnim

11.logout.php

```
<?php
```

```
session_start();
session_destroy();
header("Location: login.php");
exit;
?>
```

This code logs the user out by ending the current session and then redirects them to the login page, ensuring they can no longer access protected areas of the system.

Done by- Zarin Tasnim

Conclusion:
The Taxi Booking System with Fare Prediction provides an efficient, automated solution for urban transportation management. By integrating real-time taxi availability, automated ride assignments, and accurate fare calculations, the system addresses the limitations of traditional, manually managed taxi services. Users can conveniently book rides through an interactive interface, while the system maintains detailed records of each trip, including driver details, taxi information, distance traveled, and fare. The project demonstrates the practical application of relational database management, PHP programming, and dynamic web development to create a functional platform that enhances both user experience and taxi management efficiency. Overall, this system ensures reliable service, optimized taxi utilization, and transparent fare estimation, contributing to improved operational efficiency and passenger satisfaction.